ITERATION OPTIONS: JAVASCRIPT ARRAYS

PROG1123

FUNCTIONAL STYLE

- As a reminder, we will shape our scripting programming using the functional programming paradigm
- When it comes to iteration, functional programmers would strive to not explicitly loop at all, because loops are impure:
  - They have a counter that mutates an immutable type
- Functional programmers would, instead of explicitly looping, call either an array prototype method, or recursive functions. In this class, we will focus on array prototype methods
- Before we can get to the point of using array prototype methods in their fullest sense, we will start will learning loops, to get a feel for them
- The preferred method of looping, if you do choose to explicitly loop, would be via for … of in JS

LEAST FUNCTIONAL METHOD

- The typical for loop is usually taught first in most programming languages. It uses a counter to move through each element in the array or list-like thing.
- To stay true to our functional style, only use this type of loop if you have no other iteration alternatives

```
14  let names = ["Fred", "Donny", "Celine", "Johanna"];
15
16  for (let x = 0; x < names.length; x++){
17      console.log(names[x]);
18  }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
[Running] node "c:\Users\marsh\OneDrive\Desktop\demo.js"
Fred
Donny
Celine
Johanna
```

FOR … OF VERSUS FOR …IN

- If you just need to access each element in an array in a forward-only fashion, use the for …of method
- for … in is an alternative way to access each index in the array, not elements.

C: > Users > marsh > OneDrive > Desktop > JS demo.js > ...

```
1  let names = ["Fred", "Donny", "Celine", "Johanna"];
2  //access only elements
3  for(n of names){
4      console.log(n);
5  }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
[Running] node "c:\Users\marsh\OneDrive\Desktop\demo.js"
Fred
Donny
Celine
Johanna
```

C: > Users > marsh > OneDrive > Desktop > JS demo.js > ...

```
1  let names = ["Fred", "Donny", "Celine", "Johanna"];
2  //access only indexes
3  for(n in names){
4      console.log(n);
5  }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
[Running] node "c:\Users\marsh\OneDrive\Desktop\demo.js"
0
1
2
3
```

AN ALTERNATIVE WAY OF ACCESSING ELEMENTS

- When you call an array's values method, it returns an array of the array's values

C: > Users > marsh > OneDrive > Desktop > JS demo.js > ...

```
1  let names = ["Fred", "Donny", "Celine", "Johanna"];
2  //names.values returns an array of elements
3  for(v of names.values()){
4      console.log(v);
5  }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
[Running] node "c:\Users\marsh\OneDrive\Desktop\demo.js"
Fred
Donny
Celine
Johanna
```

AN ALTERNATIVE WAY OF ACCESSING INDEXES

- When you call an array's keys method, it returns an array of the array's indexes

```
1  let names = ["Fred", "Donny", "Celine", "Johanna"];
2  //names.keys returns an array of indexes
3  for(idx of names.keys()){
4      console.log(idx);
5  }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
[Running] node "c:\Users\marsh\OneDrive\Desktop\demo.js"
0
1
2
3
```

THE ENTRIES METHOD

- To access an element's index and value at the same time, use the entries method.
- Note that in the iteration process, you use an array of 2 variables to work with the index and value.

```javascript
let names = ["Fred", "Donny", "Celine", "Johanna"];
//names.values returns an array of elements
for([idx,v] of names.entries()){
    console.log(`the index of this element is ${idx} and its value is ${v}`);
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
[Running] node "c:\Users\marsh\OneDrive\Desktop\demo.js"
the index of this element is 0 and its value is Fred
the index of this element is 1 and its value is Donny
the index of this element is 2 and its value is Celine
the index of this element is 3 and its value is Johanna
```

EXAMPLE OF RECURSIVE METHOD

- Just in case you were interested! Here is an example of a recursive method for iteration: a function calling itself!
- In this example, the function will continue to call itself until all of the elements in the array have been spliced out.

```
14  let names = ["Fred", "Donny", "Celine", "Johanna"];
15
16  function showNames(namelist){
17      let person = namelist.splice(0,1);
18      console.log(person[0]);
19
20      if(namelist.length > 0){
21          showNames(namelist);
22      }
23  }
24  console.log(showNames(names));
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
[Running] node "c:\Users\marsh\OneDrive\Desktop\demo.js"
Fred
Donny
Celine
Johanna
```

ARRAY PROTOTYPE METHODS

- You have already been exposed to quite a few array prototype methods:
  - push
  - pop
  - splice
  - unshift
  - indexOf

- In our next lesson, we will examine array prototype methods that execute a function on each iteration over an array (map, filter, and reduce, to name a few!)